

# PLSQL

## What is PL/SQL?

PL/SQL (Procedural Language/SQL) is an extension of SQL (Structured Query Language) that allows for procedural programming constructs like loops, conditionals, and exception handling within SQL operations. Although PL/SQL is specifically an Oracle database technology, MySQL has a similar feature called MySQL Stored Programs (Procedures, Functions, and Triggers) that supports procedural extensions to SQL.

## Key Features of PL/SQL-like MySQL Extensions:

1. **Procedures and Functions:** Allow the creation of reusable code blocks.
2. **Variables:** Supports declaring and using variables in SQL operations.
3. **Control Structures:** Includes conditional statements (IF, CASE) and loops (FOR, WHILE, REPEAT).
4. **Error Handling:** Handles errors using conditions and handlers.
5. **Cursors:** Facilitates row-by-row processing of query results.

To show all the triggers in a particular database

```
SHOW TRIGGERS FROM your_database_name;
```

To show all the triggers in a particular table

```
SHOW TRIGGERS WHERE `Table` = 'table_name';
```

To show all the stored procedures within a current Database

```
SHOW PROCEDURE STATUS WHERE Db = DATABASE();
```

## PL/SQL Equivalent in MySQL: Stored Procedures and Functions

### Example 1: A Simple Stored Procedure

Task: Create a procedure to insert a new record into a table.

```
DELIMITER $$
```

```
CREATE PROCEDURE AddEmployee(
```

```
    IN emp_name VARCHAR(50),
```

```
    IN emp_dob DATE,
```

```
    IN emp_sex char(1),
```

```
    IN emp_salary DECIMAL(10, 2),
```

```
    IN emp_dept VARCHAR(100)
```

```
)
```

```
BEGIN
```

```
    INSERT INTO employees (name,dob,sex,salary,department) VALUES  
(emp_name,emp_dob,emp_sex, emp_salary,emp_dept);
```

```
END $$
```

```
DELIMITER ;
```

Usage:

```
CALL AddEmployee('Jayaa', '2068/10/26', 'F',25000.00, 'IT');
```

### Example 2: Conditional Statement in MySQL

Task: Create a stored procedure to determine if an employee is eligible for a bonus.

```
DELIMITER $$
```

```

CREATE PROCEDURE CheckBonusStatus(
    IN emp_id INT,
    OUT bonus_status VARCHAR(20)
)
BEGIN
    DECLARE emp_salary DECIMAL(10, 2);

    -- Get the employee's salary
    SELECT salary INTO emp_salary FROM employees WHERE Eld = emp_id;

    -- Check salary condition
    IF emp_salary > 40000 THEN
        SET bonus_status = 'Eligible';
    ELSE
        SET bonus_status = 'Not Eligible';
    END IF;
END $$

```

DELIMITER ;

Usage:

```

CALL CheckBonusStatus(8, @status);
SELECT @status;

```

### Example 3: Looping and Cursors

Task: Print the names of all employees with a salary greater than 40,000.

DELIMITER \$\$

```

CREATE PROCEDURE ListHighEarners()
BEGIN
    DECLARE emp_name VARCHAR(50);
    DECLARE done INT DEFAULT 0;
    DECLARE cursor_1 CURSOR FOR SELECT name FROM employees WHERE salary > 25000;
    DECLARE CONTINUE HANDLER FOR NOT FOUND SET done = 1;

    OPEN cursor_1;

    read_loop: LOOP
        FETCH cursor_1 INTO emp_name;
        IF done THEN
            LEAVE read_loop;
        END IF;
        SELECT emp_name;
    END LOOP;

    CLOSE cursor_1;
END $$

```

DELIMITER ;

Usage:

```

CALL ListHighEarners();

```

**Functions :**

**In MySQL, a function is a stored database object that:**

- **Accepts input parameters**
- **Performs a computation or operation**
- **Returns a single value**
- **Can be used within SQL statements like SELECT, WHERE, or ORDER BY**

**1)**

**DELIMITER \$\$**

```
CREATE FUNCTION get_age(dob DATE)
RETURNS INT
NOT DETERMINISTIC
BEGIN
    RETURN TIMESTAMPDIFF(YEAR, dob, CURDATE());
END $$
```

```
SELECT name, dob, TIMESTAMPDIFF(YEAR, dob, CURDATE())
AS age
```

**DELIMITER ;**

**Using function :**

**SELECT name,dob,get\_age(dob) AS AGE FROM Employees;**

**2) Function to calculate Annual Salary**

**DELIMITER \$\$**

```
CREATE FUNCTION get_annual_salary(monthly_salary DECIMAL(10,2))
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
    RETURN monthly_salary * 12;
END $$
```

**DELIMITER ;**

**Using Function :**

**SELECT name, salary, get\_annual\_salary(salary) AS annual\_salary FROM Employees;**

**3. Function to Return Employee Full Info as a String**

**DELIMITER \$\$**

```
CREATE FUNCTION get_employee_summary(eid INT)
RETURNS VARCHAR(255)
DETERMINISTIC
BEGIN
    DECLARE emp_info VARCHAR(255);

    SET emp_info = (
        SELECT CONCAT(name, ' (', department, ') earns Rs. ', salary)
        FROM Employees
        WHERE Eid = eid
    );

    RETURN emp_info;
```

**END \$\$**

**DELIMITER ;**

Using Function :

**SELECT get\_employee\_summary(3);**

**4. Function to Return Gender as Full Word**

**DELIMITER \$\$**

**CREATE FUNCTION get\_gender\_full(sex CHAR(1))**

**RETURNS VARCHAR(6)**

**DETERMINISTIC**

**BEGIN**

**RETURN CASE**

**WHEN sex = 'M' THEN 'Male'**

**WHEN sex = 'F' THEN 'Female'**

**ELSE 'Other'**

**END;**

**END \$\$**

**DELIMITER ;**

Using Function :

**SELECT name, get\_gender\_full(sex) AS gender FROM Employees;**

**Advantages of PL/SQL-like Programming in MySQL:**

- 1. Modularity:** Code can be divided into manageable blocks like procedures and functions.
- 2. Reusability:** Code can be reused across multiple SQL operations.
- 3. Improved Performance:** Reduces network traffic by executing procedural logic within the database server.
- 4. Security:** Enables controlled access to database operations using encapsulated procedures.

**Conclusion:**

Although PL/SQL is native to Oracle, MySQL provides similar functionality through its stored procedures, functions, and triggers. These allow developers to incorporate procedural logic into SQL for enhanced control and efficiency.

## **Stored Procedure vs Function – Tabular Comparison**

| <b>Feature</b>      | <b>Stored Procedure</b>                                | <b>Function</b>                                       |
|---------------------|--------------------------------------------------------|-------------------------------------------------------|
| <b>Purpose</b>      | Performs an action/task (e.g., insert, update, delete) | Computes and returns a value                          |
| <b>Return Value</b> | May or may not return a value                          | Must return exactly one value                         |
| <b>Invocation</b>   | Called using CALL statement                            | Called inside SQL queries (e.g., SELECT, WHERE, etc.) |
| <b>Use in SQL</b>   | Cannot be used directly in SQL                         | Can be used in SQL queries                            |

|                            |                                                                  |                                                               |
|----------------------------|------------------------------------------------------------------|---------------------------------------------------------------|
| <b>Statements</b>          | <b>expressions</b>                                               |                                                               |
| <b>Return Type</b>         | <b>Can return values using OUT parameters</b>                    | <b>Returns value using RETURN statement</b>                   |
| <b>Side Effects (DML)</b>  | <b>Can modify database (INSERT, UPDATE, DELETE)</b>              | <b>Not allowed to modify database (pure computation only)</b> |
| <b>Parameters</b>          | <b>Can have IN, OUT, INOUT parameters</b>                        | <b>Only IN parameters allowed</b>                             |
| <b>Transaction Control</b> | <b>Can use COMMIT/ROLLBACK</b>                                   | <b>Cannot use COMMIT or ROLLBACK</b>                          |
| <b>Usage Example</b>       | <b>CALL increase_salary(1001);</b>                               | <b>SELECT get_total_salary(1001);</b>                         |
| <b>Error Handling</b>      | <b>Can contain complex logic with handlers (DECLARE HANDLER)</b> | <b>Usually simple, not for procedural logic</b>               |

## Stored Procedures and Triggers

### Stored Procedure

A stored procedure is a set of SQL statements that can be stored in the database and executed as a program. It allows for modular programming, code reuse, and improved performance by reducing the amount of data transferred between the application and the database server. Stored procedures can include logic such as loops, conditionals, and variable declarations, making them quite powerful for database operations.

#### Benefits of Using Stored Procedures

- **Performance:** Executing a stored procedure is generally faster than executing individual SQL statements because the procedure is precompiled and stored in the database.
- **Security:** Stored procedures can help protect against SQL injection attacks.
- **Maintainability:** Encapsulating logic within a stored procedure simplifies maintenance, as changes to the logic are made in a single place.
- **Reusability:** Stored procedures can be reused across multiple applications and parts of an application.

#### Creating a Stored Procedure in MySQL

To create and use a stored procedure in MySQL, follow these steps:

1. **Connect to the MySQL Database:** Use a MySQL client such as MySQL Workbench, the command-line interface, or any other MySQL management tool.
2. **Create a Stored Procedure:** You can create a stored procedure using the `CREATE PROCEDURE` statement.

Example : Stored Procedure for Adding an employee into the Employees table Using INPUT parameters:

```
-- Create the Employees table
CREATE TABLE Employees (
```

```

id INT AUTO_INCREMENT PRIMARY KEY,
name VARCHAR(100),
dob DATE,
sex CHAR(1),
salary DECIMAL(10, 2),
department VARCHAR(100)
);

-- Insert 10 data entries into the Employees table
INSERT INTO Employees (name, dob, sex, salary, department) VALUES
('John Doe', '1985-05-15', 'M', 60000.00, 'HR'),
('Jane Smith', '1990-10-25', 'F', 75000.00, 'Finance'),
('Alice Johnson', '1988-07-12', 'F', 68000.00, 'IT'),
('Bob Brown', '1982-03-18', 'M', 72000.00, 'Marketing'),
('Carol White', '1995-11-30', 'F', 55000.00, 'Sales'),
('David Wilson', '1992-09-23', 'M', 82000.00, 'Operations'),
('Eva Green', '1987-04-09', 'F', 59000.00, 'HR'),
('Frank Black', '1993-06-16', 'M', 91000.00, 'Finance'),
('Grace Clark', '1986-02-28', 'F', 65000.00, 'IT'),
('Henry Lewis', '1989-08-21', 'M', 78000.00, 'Marketing');

```

DELIMITER \$\$

```

CREATE PROCEDURE AddEmployee(
    IN emp_name VARCHAR(100),
    IN emp_dob DATE,
    IN emp_sex CHAR(1),
    IN emp_salary DECIMAL(10,2),
    IN emp_department VARCHAR(100)
)
BEGIN
    INSERT INTO employees (name, dob, sex, salary, department)
    VALUES (emp_name, emp_dob, emp_sex, emp_salary, emp_department);
END $$

```

DELIMITER ;

Now you can insert any number of employees without using sql query but just by calling stored procedure with parameters

```

CALL AddEmployee('Jayaram', '1986-06-26', 'M', 60000.00, 'IT');
CALL AddEmployee('Diwas', '1996-09-13', 'M', 80000.00, 'Engineering');

```

Example 2 : Accessing Data from database using Stored Procedure with OUTPUT Parameters

```

DELIMITER $$
CREATE PROCEDURE GetEmployeesDetail(
    IN emp_id INT,
    OUT emp_name VARCHAR(100),
    OUT emp_department VARCHAR(100),
    OUT emp_salary DECIMAL(10,2)
)

```

```

)
BEGIN
    SELECT name, department, salary
    INTO emp_name, emp_department, emp_salary
    FROM employees
    WHERE EId = emp_id;
END $$

```

DELIMITER ;

Now you can access the employee's Name, Department and salary by calling stored procedure

```

SET @emp_id = 5;
SET @emp_name = '';
SET @emp_department = '';
SET @emp_salary = 0;

```

-- Call the stored procedure

```
CALL GetEmployeeDetails(@emp_id, @emp_name, @emp_department, @emp_salary);
```

-- Display the output values

```
SELECT @emp_name, @emp_department, @emp_salary;
```

Example 3 :

i) Increase the salary of employees who belongs to IT department.

DELIMITER \$\$

```
CREATE PROCEDURE UpdateITDeptSalary()
```

```
BEGIN
```

-- Declare any necessary variables

```
DECLARE emp_count INT DEFAULT 0;
```

```
DECLARE updated_count INT DEFAULT 0;
```

-- Count the number of employees in the IT department before the update

```
SELECT COUNT(*) INTO emp_count FROM employees WHERE department = 'IT';
```

```
END $$
```

DELIMITER ;

-- Update the salary by 15% for all employees in the IT department

```
UPDATE employees
```

```
SET salary = salary * 1.15
```

```
WHERE department = 'IT';
```

-- Count the number of employees updated

```
SELECT ROW_COUNT() INTO updated_count;
```

ii) Updating Salary of Employee whose ID is given

DELIMITER \$\$

```

CREATE PROCEDURE UpdateSalary(
    IN emp_id INT,
    IN performance_rating INT,
    OUT old_salary DECIMAL(10, 2),
    OUT new_salary DECIMAL(10, 2)
)
BEGIN
    -- Declare a variable to store the current salary
    DECLARE current_salary DECIMAL(10, 2);

    -- Retrieve the current salary of the employee
    SELECT salary INTO current_salary FROM employees WHERE id = emp_id;

    -- Set the old salary output parameter
    SET old_salary = current_salary;

    -- Calculate the new salary based on performance rating
    IF performance_rating = 1 THEN
        SET new_salary = current_salary * 1.15;
    ELSEIF performance_rating = 2 THEN
        SET new_salary = current_salary * 1.10;
    ELSE
        SET new_salary = current_salary * 1.05;
    END IF;

    -- Update the employee's salary
    UPDATE employees
    SET salary = new_salary
    WHERE id = emp_id;
END $$

DELIMITER ;

SET @emp_id = 2;
SET @performance_rating = 2;
SET @old_salary = 0.0;
SET @new_salary = 0.0;

-- Call the stored procedure
CALL UpdateSalary(@emp_id, @performance_rating, @old_salary, @new_salary);

-- Display the results
SELECT
    e.id AS 'Employee ID',
    e.name AS 'Employee Name',
    @old_salary AS 'Old Salary',
    @new_salary AS 'New Salary'

```



```
FROM employees e
WHERE e.id = @emp_id;
```

Example 4 : delete an employee by id  
DELIMITER \$\$

```
CREATE PROCEDURE DeleteEmployeeById(
    IN emp_id INT
)
BEGIN
    -- Delete the employee with the given ID
    DELETE FROM employees WHERE id = emp_id;
END $$
```

DELIMITER ;

```
CALL DeleteEmployeeById(12);
```

### Trigger

A trigger in MySQL is a set of instructions that are automatically executed (or "triggered") when a specified event occurs in the database. Triggers are used to perform automated actions in response to data modifications such as INSERT, UPDATE, or DELETE operations on a specified table.

### Types of Triggers in MySQL

MySQL supports the following types of triggers:

1. **BEFORE INSERT**: Triggered before a new row is inserted into a table.
2. **AFTER INSERT**: Triggered after a new row is inserted into a table.
3. **BEFORE UPDATE**: Triggered before an existing row is updated in a table.
4. **AFTER UPDATE**: Triggered after an existing row is updated in a table.
5. **BEFORE DELETE**: Triggered before an existing row is deleted from a table.
6. **AFTER DELETE**: Triggered after an existing row is deleted from a table.

### Syntax of Trigger

```
CREATE TRIGGER trigger_name
    {BEFORE | AFTER} {INSERT | UPDATE | DELETE}
    ON table_name
    FOR EACH ROW
BEGIN
    -- Trigger logic
END;
```

### Sample Table

```
CREATE TABLE Employees (
    Eid INT PRIMARY KEY AUTO_INCREMENT,
    name VARCHAR(100),
    dob DATE,
    sex CHAR(1),
```

```
salary DECIMAL(10, 2),
department VARCHAR(100)
);
```

#### Example 1) Insert Trigger

i) Creating a trigger that checks if salary is less than 15000 and if so then displays the message

```
DELIMITER $$
```

```
CREATE TRIGGER before_insert_check_salary
BEFORE INSERT ON Employees
FOR EACH ROW
BEGIN
    IF NEW.salary < 15000 THEN
        SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Salary must not be less than 15000';
    END IF;
END $$
```

```
DELIMITER ;
```

```
insert into Employees(name,department,sex,dob,salary) values("Diwas","Geo","M","2003-05-12","12000")
```

```
insert into Employees(name,department,sex,dob,salary) values("Diwas","Geo","M","2003-05-12","16000")
```

```
Insert into Employees(name, dob, sex, salary, department) values("Rudrakshya Mishra", "2012-02-12", "F", 12000, "No-Dept");
```

#### ii) Insert Log record

```
CREATE TABLE log_insert_Employees (
    log_id INT PRIMARY KEY AUTO_INCREMENT,
    emp_id INT,
    insert_time DATETIME DEFAULT CURRENT_TIMESTAMP,
    user VARCHAR(255)
);
```

```
DELIMITER $$
```

```
CREATE TRIGGER after_insert_log_insert_employees
AFTER INSERT ON employees
FOR EACH ROW
BEGIN
    INSERT INTO log_insert_Employees (emp_id,user)
```

```
VALUES (NEW.EId,USER());  
END $$
```

DELIMITER ;

Now inserting two records

```
Insert into Employees(name, dob, sex, salary, department) values("Durga", "1986-02-12", "F",  
20000, "Mgmt"), ("Jayaram", "1966-03-22", "M", 30000, "IT");
```

Now lets check the log\_insert\_Employees table

```
Select * from log_insert_Employees;
```

Example 2) Update Trigger

i) Before Update check if salary is updated to less than 15000

DELIMITER \$\$

```
CREATE TRIGGER before_update_salary_check  
BEFORE UPDATE ON Employees  
FOR EACH ROW  
BEGIN  
    -- Check if the new salary is less than 15,000  
    IF NEW.salary < 15000 THEN  
        -- Signal an error and prevent the update  
        SIGNAL SQLSTATE '45000'  
        SET MESSAGE_TEXT = 'Salary must be at least 15,000.';  
    END IF;  
END $$
```

DELIMITER ;

```
Update Employees set salary = 12000 where id = 11;
```

ii) Update log record

Creating table for log record

```
CREATE TABLE log_update_Employees (  
    log_id INT AUTO_INCREMENT PRIMARY KEY,  
    emp_id INT,  
    old_name VARCHAR(100),  
    old_dob DATE,  
    old_sex CHAR(1),  
    old_salary DECIMAL(10, 2),  
    old_department VARCHAR(100),
```

```

    new_name VARCHAR(100),
    new_dob DATE,
    new_sex CHAR(1),
    new_salary DECIMAL(10, 2),
    new_department VARCHAR(100),
    update_time DATETIME DEFAULT CURRENT_TIMESTAMP,
    user VARCHAR(255)
);

```

Creating Trigger for after update  
DELIMITER \$\$

```

CREATE TRIGGER after_update_log_employees
AFTER UPDATE ON Employees
FOR EACH ROW
BEGIN
    INSERT INTO log_update_Employees (emp_id, old_name, old_dob, old_sex, old_salary,
old_department, new_name, new_dob, new_sex, new_salary, new_department, user)
    VALUES
    (OLD.Eid, OLD.name, OLD.dob, OLD.sex, OLD.salary, OLD.department, NEW.name, NEW.dob,
NEW.sex, NEW.salary, NEW.department, USER());
END $$

DELIMITER ;

```

Update Employees set salary = 80000, dob="2080-12-12" where department LIKE "HR";

Example 3) Delete Trigger

ii) Creating Table for storing deleted record

```

CREATE TABLE log_delete_Employees (
    log_id INT AUTO_INCREMENT PRIMARY KEY,
    emp_id INT,
    delete_time DATETIME DEFAULT CURRENT_TIMESTAMP,
    user VARCHAR(255)
);

```

DELIMITER \$\$

```

CREATE TRIGGER after_delete_log_employees
AFTER DELETE ON Employees
FOR EACH ROW
BEGIN
    INSERT INTO log_delete_Employees (emp_id, `user`)
    VALUES (OLD.Eid, USER());
END $$
DELIMITER ;

```

Delete from Employees where Eid = 7;

## Stored Procedures

### Definition

- A **stored procedure** is precompiled SQL code stored in the database that can be executed by applications or other SQL scripts. It allows developers to encapsulate logic within the database, reducing redundant code.

### Advantages

1. **Improved Performance:**
  - Executes directly on the database server, reducing network traffic.
2. **Reusability:**
  - Encapsulates frequently used SQL logic, reducing code duplication.
3. **Security:**
  - Provides a controlled way to access and manipulate data.
4. **Simplifies Complex Logic:**
  - Handles complex calculations or operations within the database.
5. **Reduced Network Load:**
  - Executes multiple operations on the server side without frequent client-server communication.

### Real-World Scenario-Based Examples

#### Example 1: E-Commerce Platform - Retrieve Customer Orders

##### Scenario:

- An e-commerce platform retrieves all orders for a specific customer.

##### Stored Procedure:

```
CREATE PROCEDURE GetCustomerOrders(IN customerID INT)
BEGIN
    SELECT * FROM orders WHERE customer_id = customerID;
END;
```

##### Outcome:

- Reduces redundancy by encapsulating the SQL logic for order retrieval into a single reusable procedure.

#### Example 2: Banking System - Monthly Interest Calculation

##### Scenario:

- A bank calculates monthly interest for savings accounts.

##### Stored Procedure:

```
CREATE PROCEDURE CalculateInterest()
```

**BEGIN**

**UPDATE accounts**

**SET balance = balance + (balance \* interest\_rate / 12)**

**WHERE account\_type = 'Savings';**

**END;**

**Outcome:**

- Automates the interest calculation process for all savings accounts, saving time and ensuring consistency.

### **Example 3: Employee Management - Bulk Salary Update**

**Scenario:**

- HR needs to apply a 10% salary increment to all employees in a specific department.

**Stored Procedure:**

```
CREATE PROCEDURE UpdateSalaryByDepartment(IN deptName VARCHAR(50), IN incrementPercentage FLOAT)
```

```
BEGIN
```

```
UPDATE employees
```

```
SET salary = salary + (salary * incrementPercentage / 100)
```

```
WHERE department = deptName;
```

```
END;
```

**Outcome:**

- Simplifies bulk updates, making the process efficient and error-free.

### **Example 4: Healthcare - Fetch Patient Details**

**Scenario:**

- A hospital retrieves detailed information for a patient during an appointment.

**Stored Procedure:**

```
CREATE PROCEDURE GetPatientDetails(IN patientID INT)
```

```
BEGIN
```

```
SELECT name, age, medical_history FROM patients WHERE id = patientID;
```

```
END;
```

**Outcome:**

- Provides quick access to patient data, enhancing the doctor-patient interaction.

### **Example 5: Inventory Management - Low Stock Alert**

**Scenario:**

- A warehouse generates a report of items running low on stock.

**Stored Procedure:**

```
CREATE PROCEDURE GetLowStockItems(IN threshold INT)  
BEGIN  
    SELECT item_name, stock_quantity FROM inventory WHERE stock_quantity < threshold;  
END;
```

**Outcome:**

- Automates low-stock reporting, aiding in timely restocking.

**Applications of Stored Procedures**

1. Batch processing (e.g., interest calculation, payroll processing).
2. Business logic enforcement (e.g., discount calculations).
3. Complex report generation (e.g., sales summaries).
4. Data validation before insertion/updation.

**4.2 Triggers****Definition**

- A **trigger** is an automated database action executed in response to specific events such as INSERT, UPDATE, or DELETE. It is used to enforce business rules, maintain data integrity, or log database activity.

**Advantages**

1. **Automation:**
  - Executes logic automatically when specified events occur.
2. **Data Integrity:**
  - Enforces constraints and business rules.
3. **Logging:**
  - Tracks changes in database tables for auditing.
4. **Error Prevention:**
  - Automatically prevents invalid data from being inserted.
5. **Consistency:**
  - Maintains consistency across dependent tables.

**Real-World Scenario-Based Examples**

### **Example 1: E-Commerce - Order Logging**

#### **Scenario:**

- Log every new order into a separate logs table for auditing purposes.

#### **Trigger:**

```
CREATE TRIGGER AfterInsertOrder  
AFTER INSERT ON orders  
FOR EACH ROW  
BEGIN  
    INSERT INTO logs (action, order_id, timestamp) VALUES ('Order Inserted', NEW.order_id,  
    NOW());  
END;
```

#### **Outcome:**

- Ensures every new order is logged automatically, aiding in audit and analysis.

### **Example 2: Banking System - Transaction Audit**

#### **Scenario:**

- Track every deposit or withdrawal transaction in a separate audit table.

#### **Trigger:**

```
CREATE TRIGGER AuditTransaction  
AFTER UPDATE ON accounts  
FOR EACH ROW  
BEGIN  
    INSERT INTO transaction_audit (account_id, old_balance, new_balance, timestamp)  
    VALUES (NEW.account_id, OLD.balance, NEW.balance, NOW());  
END;
```

#### **Outcome:**

- Maintains a detailed history of account changes, improving transparency.

### **Example 3: University System - Enforce Course Capacity**

#### **Scenario:**

- Prevent students from registering for courses that have reached their maximum capacity.

#### **Trigger:**

```
CREATE TRIGGER PreventOverRegistration  
BEFORE INSERT ON enrollments
```



**FOR EACH ROW**

**BEGIN**

**DECLARE current\_capacity INT;**

**SELECT COUNT(\*) INTO current\_capacity FROM enrollments WHERE course\_id = NEW.course\_id;**

**IF current\_capacity >= (SELECT max\_capacity FROM courses WHERE id = NEW.course\_id) THEN**

**SIGNAL SQLSTATE '45000' SET MESSAGE\_TEXT = 'Course is full';**

**END IF;**

**END;**

**Outcome:**

- Prevents over-enrollment, ensuring compliance with course limits.

#### **Example 4: Healthcare - Automatic Age Calculation**

**Scenario:**

- Automatically calculate and update a patient's age upon record insertion.

**Trigger:**

**CREATE TRIGGER CalculateAgeBeforeInsert**

**BEFORE INSERT ON patients**

**FOR EACH ROW**

**BEGIN**

**SET NEW.age = TIMESTAMPDIFF(YEAR, NEW.date\_of\_birth, CURDATE());**

**END;**

**Outcome:**

- Ensures the age field is always accurate and up-to-date.

#### **Example 5: Inventory Management - Stock Validation**

**Scenario:**

- Prevent orders for items that are out of stock.

**Trigger:**

**CREATE TRIGGER ValidateStockBeforeInsert**

**BEFORE INSERT ON order\_items**

**FOR EACH ROW**

**BEGIN**

**DECLARE available\_stock INT;**

```
SELECT stock_quantity INTO available_stock FROM inventory WHERE item_id =  
NEW.item_id;  
  
IF available_stock < NEW.quantity THEN  
    SIGNAL SQLSTATE '45000' SET MESSAGE_TEXT = 'Insufficient stock';  
END IF;  
  
END;
```

#### **Outcome:**

- Ensures stock levels are not exceeded, maintaining inventory accuracy.

#### **Applications of Triggers**

1. Logging changes (e.g., transaction history).
2. Enforcing constraints (e.g., capacity limits, stock availability).
3. Automating calculations (e.g., age, tax).
4. Sending notifications (e.g., email alerts on certain actions).
5. Synchronizing dependent tables (e.g., cascading updates or deletions).

#### **Key Takeaways**

1. **Stored Procedures:**
  - Encapsulate reusable logic within the database, simplifying complex operations and improving performance.
2. **Triggers:**
  - Automate actions in response to database events, enforcing consistency and reducing manual intervention.
3. **Real-World Applications:**
  - Found in domains like e-commerce, banking, healthcare, education, and inventory management.
4. **Best Practices:**
  - Avoid excessive use of triggers to prevent unintended complexities.
  - Combine stored procedures and triggers judiciously for optimal database management.